Q1. Explain the Breadth First Search (BFS) and Depth First Search (DFS) graph traversal algorithms in detail.

Answer:

Graph traversal is the process of visiting every vertex (node) in a graph exactly once. The two primary methods are BFS and DFS.

1. Breadth First Search (BFS)

BFS is a traversal algorithm that explores the graph layer-by-layer. It starts at a source node and visits all its immediate neighbors before moving to the next level neighbors.

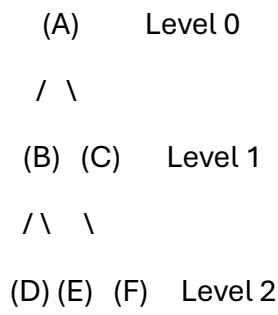
- **Principle:** Level-order traversal.
- **Data Structure Used: Queue** (FIFO - First In, First Out).
- **Application:** Finding the shortest path in unweighted graphs, peer-to-peer networks.

Algorithm Steps:

1. Initialize a boolean array visited[] to false.
2. Enqueue the starting vertex S and mark it as visited.
3. While the **Queue** is not empty:
    - **Dequeue** a vertex V and print it.
    - For every adjacent vertex U of V:
        - If U is not visited, mark it as visited and **Enqueue** it.

Block Diagram (Traversal Flow):

Plaintext



Queue Operations:

1. Enqueue A.

2. Dequeue A, Enqueue B, C.

3. Dequeue B, Enqueue D, E.

4. Dequeue C, Enqueue F.

5. Dequeue D, E, F (empty).

Output: A -> B -> C -> D -> E -> F

2. Depth First Search (DFS)

DFS is a traversal algorithm that explores as deep as possible along each branch before backtracking. It follows a single path until it hits a dead-end, then returns to the last junction.

- **Principle:** Backtracking.
- **Data Structure Used: Stack** (LIFO) or **Recursion**.
- **Application:** Solving mazes, topological sorting, detecting cycles.

Algorithm Steps:

1. Push the starting vertex S onto the **Stack** (or call recursive function).
2. Mark S as visited and print it.
3. For every adjacent vertex U of S:
    - If U is not visited, recursively call DFS on U.
4. If a node has no unvisited neighbors, **backtrack** to the previous node.

Block Diagram (Traversal Flow):

Using the same graph as above:

Plaintext

```
   (A)
  /   \
(B)   (C)
```

/ \ \

(D) (E) (F)

Stack/Recursion Flow:

1. Start at A.

2. Go Deep to B.

3. Go Deep to D. (D is leaf, Backtrack to B)

4. Go Deep to E. (E is leaf, Backtrack to B, then A)

5. Go Deep to C.

6. Go Deep to F.

Output: A -> B -> D -> E -> C -> F

Q2. Explain the concept of a two-dimensional (2D) array and describe in detail how to calculate the address of an element in memory.

Answer:

1. Concept of 2D Arrays

A Two-Dimensional (2D) array is a collection of elements organized in a grid structure consisting of **Rows** and **Columns**. It is essentially an "array of arrays."

- **Declaration (C):** int matrix[Rows][Cols];
- **Visual Structure:**

Plaintext

Col 0 Col 1 Col 2

Row 0: [0,0] [0,1] [0,2]

Row 1: [1,0] [1,1] [1,2]

- **Memory Storage:** Computer memory is linear (1D). Therefore, 2D arrays must be "linearized" to be stored, using either Row-Major or Column-Major order.

2. Address Calculation

To find the physical memory address of an element $A[i][j]$, we use the following parameters:

- BA: Base Address (Address of first element).
- W: Size of each element in bytes.
- N: Number of Columns.
- M: Number of Rows.
- L_r, L_c: Lower bound of rows and columns (usually 0).

A. Row-Major Order (Row by Row)

Elements are stored row by row. To reach element $[i][j]$, we must skip i full rows.

$$Address(A[i][j]) = BA + W \times [N \times (i - L_r) + (j - L_c)]$$

B. Column-Major Order (Column by Column)

Elements are stored column by column. To reach element $[i][j]$, we must skip j full columns.

$$Address(A[i][j]) = BA + W \times [(i - L_r) + M \times (j - L_c)]$$

Q3. Define a stack and explain its operations. Discuss how a stack is used to convert an infix expression to a postfix expression.

Answer:

1. Definition

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. Elements are added and removed from only one end, called the **Top**.

2. Operations

- **Push(x):** Adds element x to the Top. (Check for Overflow).
- **Pop():** Removes and returns the Top element. (Check for Underflow).
- **Peek():** Views the Top element without removing it.

3. Infix to Postfix Conversion

A stack is used to hold operators ($+, -, *, /$) and parentheses to ensure correct precedence without using brackets in the final output.

Algorithm:

1. Scan expression left to right.
2. If operand: Print it.
3. If operator: Pop from stack to output while stack top has $\ge$ precedence. Then Push current operator.
4. If (: Push to stack.
5. If): Pop to output until (is found.

Step-by-Step Example:

Input: $A + B * (C - D)$

Symbol	**Stack Status**	**Output (Postfix)**	**Reason**
A	Empty	**A**	Operand
+	+	**A**	Push operator
B	+	**A B**	Operand
*****	+ *	**A B**	* > +, Push *
(	+ * (	**A B**	Push (
C	+ * (	**A B C**	Operand
-	+ * (-	**A B C**	Push -
D	+ * (-	**A B C D**	Operand

Symbol	Stack Status	Output (Postfix)	Reason
)	+ *	**A B C D -**	Pop until (
End	Empty	**A B C D - * +**	Pop all

Result: $A B C D - * +$

Q4. What is a linked list? Explain the structure of a singly linked list. Describe basic operations.

Answer:

1. Definition

A **Linked List** is a dynamic linear data structure where elements (Nodes) are stored in non-contiguous memory locations. Each node points to the next node.

2. Structure of Singly Linked List

Each node contains two fields:

1. **Data:** The value.
2. **Next:** A pointer to the address of the next node.

Block Diagram:

Plaintext

[Head] --> [Data | Next] --> [Data | Next] --> [Data | NULL]

Node 1 Node 2 Node 3

3. Basic Operations

- Traversal:

Start at Head, print data, move to next pointer, repeat until NULL.

- **Insertion (at Beginning):**

Plaintext

NewNode -> next = Head;

Head = NewNode;

Diagram: [New] -> [Old Head] ...

- **Deletion (from Beginning):**

Plaintext

Temp = Head;

Head = Head -> next;

Free(Temp);

Diagram: [Head] (moves forward) --> [2nd Node]

Q5. Define a binary tree and explain the terms: external node and internal node.

Answer:

1. Binary Tree Definition

A **Binary Tree** is a hierarchical structure where every node has **at most two children** (Left Child and Right Child).

2. Terminology

A. Internal Node (Inner Node)

- **Definition:** Any node that has **at least one child**.
- **Function:** These nodes represent the branching logic or operators in an expression tree.
- **Example:** In the tree below, A and B are internal.

B. External Node (Leaf Node)

- **Definition:** A node that has **no children** (Left and Right pointers are NULL).
- **Function:** These nodes represent the data or operands.
- **Example:** In the tree below, D, E, and C are external.

Diagram:

Plaintext

(A) <-- Internal (Root)

/ \

(B) (C) <-- C is External (Leaf)

/ \

(D) (E) <-- D, E are External (Leaves)

^

|

B is Internal

Q6. Convert the following infix expression into prefix form: (A+B)*(C-D)/E

Answer:

To convert to Prefix (Polish Notation), we move operators before their operands.

Expression: $(A+B)*(C-D)/E$

Step 1: Resolve Parentheses

- $(A+B) \rightarrow +AB$
- $(C-D) \rightarrow -CD$

Intermediate: $(+AB) * (-CD) / E$

Step 2: Resolve Multiplication (*)

- Precedence of * and / is equal, associativity is Left-to-Right.
- Operands: (+AB) and (-CD)
- Apply *: $* + A B - C D$

Intermediate: $[* + A B - C D] / E$

Step 3: Resolve Division (/)

- Operands: [*+AB-CD] and E
- Apply /: $/ * + A B - C D E$

Final Prefix Expression:

$$/ * + A B - C D E$$

Tree Verification:

Plaintext

```
      /
     / \
    * E
   / \
  + -
 / \ / \
A B C D
```

Pre-order traversal of this tree yields the derived answer.
